

Tool Use

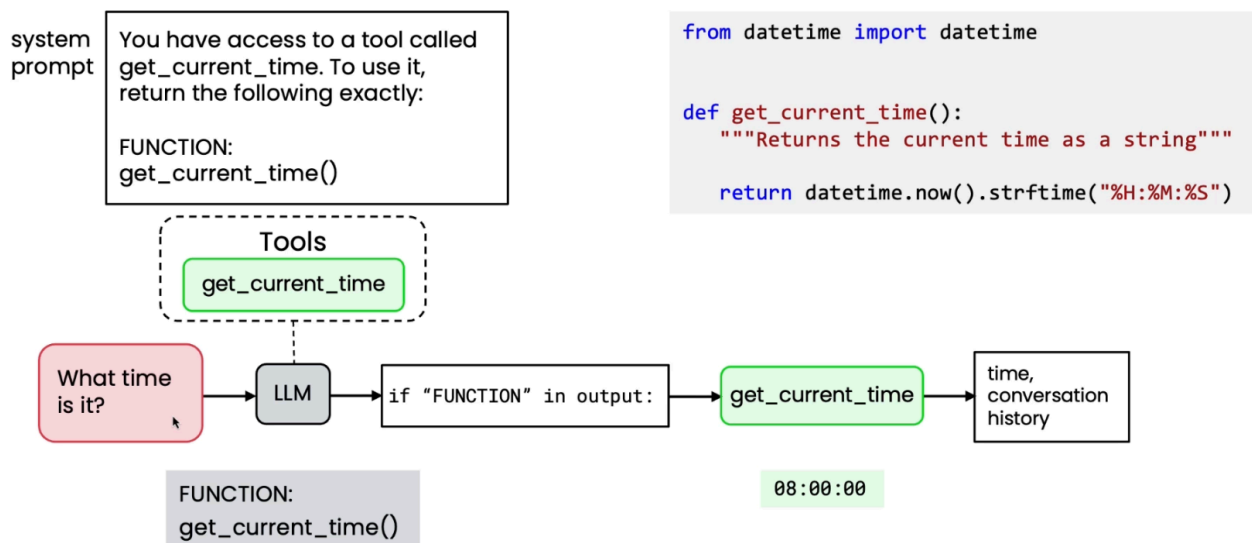
Using the proper tool is beneficial for the workflow, and we can get the desired outcome.

Examples

Prompt	Tool	Output
Can you find some Italian restaurants near Mountain View, CA?	<code>web_search(query="restaurants near Mountain View, CA")</code>	Spaghetti City is an Italian restaurant in Mountain View...
Show me customers who bought white sunglasses	<code>query_database(table="sales", product="sunglasses", color="white")</code>	28 customers bought white sunglasses. Here they are...
How much money will I have after 10 years if I deposit \$500 at 5% interest?	<code>interest_calc(principal=500, interest_rate=5, years=10)</code> OR <code>eval("500 * (1 + 0.05) ** 10")</code>	\$814.45

Create a Tool

Prompting an LLM to use tools



system
prompt

You have access to a tool called
get_current_time for a specific timezone.
To use it, return the following exactly:

FUNCTION:
get_current_time("timezone")

Tools

get_current_time

What time
is it in New
Zealand?

LLM

if "FUNCTION" in output:

get_current_time(
"Pacific/Auckland")

time,
conversation
history

LLM

FUNCTION:

get_current_time("Pacific/Auckland")

04:00:00

It's 4am.

Tool syntax

```
from datetime import datetime

def get_current_time():

    """Returns the current time as a string"""

    return datetime.now().strftime("%H:%M:%S")
```

```
import aisuite as ai
client = ai.Client()

response = client.chat.completions.create(
    model="openai:gpt-4o",
    messages=messages,
    tools=[get_current_time],
    max_turns=5
)
```

JSON Schema

```
from datetime import datetime
from zoneinfo import ZoneInfo

def get_current_time(timezone):
    """Returns current time for the given time zone"""
    timezone = ZoneInfo(timezone)
    return datetime.now(timezone).strftime("%H:%M:%S")
```

```
import aisuite as ai
client = ai.Client()

response = client.chat.completions.create(
    model="openai:gpt-4o",
    messages=messages,
    tools=[get_current_time],
    max_turns=5
)
```

```
tools = [{ "type": "function",
            "function": { "name": "get_current_time",
                          "description": "Returns current time for the given time zone.",
                          "parameters": {
                              "timezone": {
                                  "type": "string",
                                  "description": "The IANA time zone string, e.g., 'America/New_York' or 'Pacific/Auckland'."
                              }
                          }
            },
          }
]
```

Key term:

- `response.choices[0]` refers to the first completion choice in the API response array.
- **Structure:** The `choices` key in an API response returns an array (or list) of potential text completions.
- **The [0] Index:** The `[0]` represents the first item in that array. If you only request one completion, this is where the generated text lives.

- **Multiple Choices:** If you set a parameter like `n=5` in your API request, the `choices` array will contain five different completions. In that case, `response.choices[0]` would be the first, `response.choices[1]` the second, and so on.
- **Accessing Text:** To get the actual text output, you typically access `response.choices[0].text` (in older APIs) or `response.choices[0].message.content` (in newer chat-based APIs).

```
import json
import display_functions
from dotenv import load_dotenv
_ = load_dotenv()

import aisuite as ai

# Create an instance of the AISuite client
client = ai.Client()

from datetime import datetime

def get_current_time():
    """
    Returns the current time as a string.
    """
    return datetime.now().strftime("%H:%M:%S")

get_current_time()

# Message structure
prompt = "What time is it?"
messages = [
    {
        "role": "user",
        "content": prompt,
    }
]

response = client.chat.completions.create(
    model="openai:gpt-4o",
    messages=messages,
    tools=[get_current_time],
    max_turns=5
)
```

```

# See the LLM response
print(response.choices[0].message.content)

response2 = None

# Create a condition in case tool_calls is in response object
if response.choices[0].message.tool_calls:
    # Pull out the specific tool metadata from the response
    tool_call = response.choices[0].message.tool_calls[0]
    args = json.loads(tool_call.function.arguments)

    # Run the tool locally
    tool_result = get_current_time()

    # Append the result to the messages list
    messages.append(response.choices[0].message)
    messages.append({
        "role": "tool", "tool_call_id": tool_call.id, "content": str(tool_result)
    })

    # Send the list of messages with the newly appended results back to the LLM
    response2 = client.chat.completions.create(
        model="openai:gpt-4o",
        messages=messages,
        tools=tools,
    )

    print(response2.choices[0].message.content)

```

We add three more tools

- Weather tool (get_weather_from_ip)
- File warning tool (write_txt_file)
- QR code generator (generate_qr_code)

```

import requests
import qrcode
from qrcode.image.styledpil import StyledPILImage

def get_weather_from_ip():
    """
    Gets the current, high, and low temperature in Fahrenheit for the user's
    location and returns it to the user.

```

```

"""
# Get location coordinates from the IP address
lat, lon = requests.get('https://ipinfo.io/json').json()['loc'].split(',')

# Set parameters for the weather API call
params = {
    "latitude": lat,
    "longitude": lon,
    "current": "temperature_2m",
    "daily": "temperature_2m_max,temperature_2m_min",
    "temperature_unit": "fahrenheit",
    "timezone": "auto"
}

# Get weather data
weather_data = requests.get("https://api.open-meteo.com/v1/forecast",
params=params).json()

# Format and return the simplified string
return (
    f"Current: {weather_data['current']['temperature_2m']}°F, "
    f"High: {weather_data['daily']['temperature_2m_max'][0]}°F, "
    f"Low: {weather_data['daily']['temperature_2m_min'][0]}°F"
)

# Write a text file
def write_txt_file(file_path: str, content: str):
    """
    Write a string into a .txt file (overwrites if exists).
    Args:
        file_path (str): Destination path.
        content (str): Text to write.
    Returns:
        str: Path to the written file.
    """
    with open(file_path, "w", encoding="utf-8") as f:
        f.write(content)
    return file_path

# Create a QR code
def generate_qr_code(data: str, filename: str, image_path: str):
    """Generate a QR code image given data and an image path.

    Args:
        data: Text or URL to encode
        filename: Name for the output PNG file (without extension)
        image_path: Path to the image to be used in the QR code
    """
    qr = qrcode.QRCode(error_correction=qrcode.constants.ERROR_CORRECT_H)

```

```

qr.add_data(data)

img = qr.make_image(image_factory=StyledPILImage,
embedded_image_path=image_path)
output_file = f"{filename}.png"
img.save(output_file)

return f"QR code saved as {output_file} containing: {data[:50]}..."

```

using the new tools

```

prompt = "Can you get the weather for my location?"

response = client.chat.completions.create(
    model="openai:o4-mini",
    messages=[{"role": "user", "content": (
        prompt
    )}],
    tools=[
        get_current_time,
        get_weather_from_ip,
        write_txt_file,
        generate_qr_code
    ],
    max_turns=5
)

display_functions.pretty_print_chat_completion(response)

```

```

prompt = "Can you make a txt note for me called reminders.txt that reminds me to
call Daniel tomorrow at 7PM?"

```

```

response = client.chat.completions.create(
    model="openai:o4-mini",
    messages=[{"role": "user", "content": (
        prompt
    )}],
    tools=[
        get_current_time,
        get_weather_from_ip,
        write_txt_file,
        generate_qr_code
    ],
    max_turns=5
)

display_functions.pretty_print_chat_completion(response)

```

**LLM Action:** write_txt_file

```
{  
  "file_path": "reminders.txt",  
  "content": "Reminder: Call Daniel tomorrow at 7PM."  
}
```

**Tool Response:** write_txt_file

"reminders.txt"

**Final Assistant Message:**

I've created your note "reminders.txt" with the reminder: "Call Daniel tomorrow at 7PM." Let me know if there's anything else you need!

**Tool Sequence:**

write_txt_file

open the text file and read the contents to make sure that it now exists.

```
with open('reminders.txt', 'r') as file:  
    contents = file.read()  
    print(contents)
```

Create the QR code.

```
`` prompt = "Can you make a QR code for me using my company's logo that goes  
to www.deeplearning.ai? The logo is located at dl_logo.jpg`. You can call it  
dl_qr_code."
```

```
response = client.chat.completions.create(  
model="openai:o4-mini",  
messages=[{"role": "user", "content": (  
prompt  
)}],  
tools=[  
get_current_time,  
get_weather_from_ip,  
write_txt_file,  
generate_qr_code  
],
```

```
max_turns=5
)

display_functions.pretty_print_chat_completion(response)
...
```

 LLM Action: generate_qr_code

```
{
  "data": "www.deeplearning.ai",
  "filename": "dl_qr_code",
  "image_path": "dl_logo.jpg"
}
```

 Tool Response: generate_qr_code

"QR code saved as dl_qr_code.png containing: www.deeplearning.ai..."

 Final Assistant Message:

Your QR code has been generated and saved as dl_qr_code.png, embedding your company's logo and linking to www.deeplearning.ai. Let me know if you need anything else!

 Tool Sequence:

generate_qr_code

```
from IPython.display import Image, display

# Display image directly
Image('dl_qr_code.png')
```

Output:

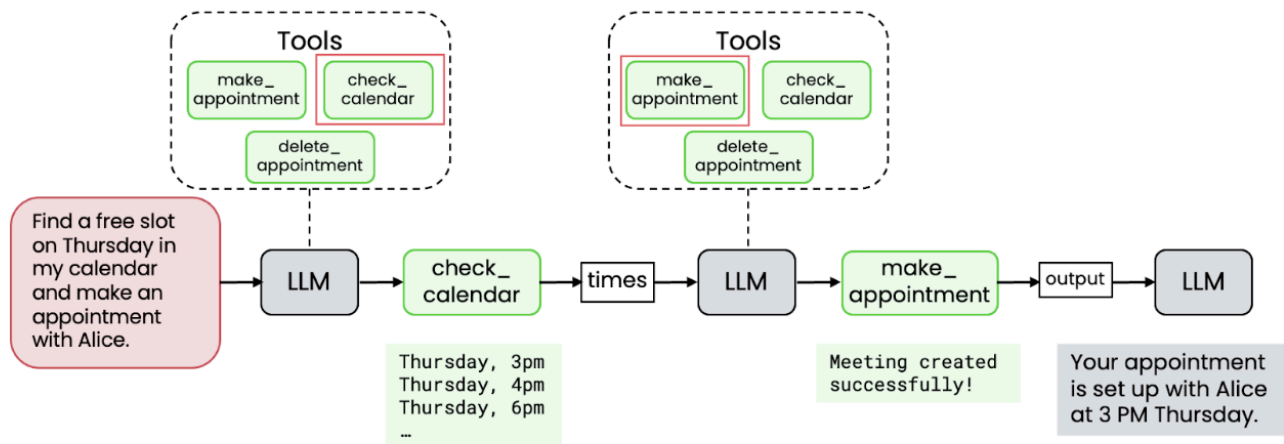


Use multiple tools at once

```
prompt = "Can you help me create a qr code that goes to www.deeplearning.com from  
the image dl_logo.jpg? Also write me a txt note with the current weather please."  
  
response = client.chat.completions.create(  
    model="openai:o4-mini",  
    messages=[{"role": "user", "content": (  
        prompt  
    )}],  
    tools=[  
        get_weather_from_ip,  
        get_current_time,  
        write_txt_file,  
        generate_qr_code  
    ],  
    max_turns=10  
)  
  
display_functions.pretty_print_chat_completion(response)
```

Email Assistant workflow

Multiple tools



2. Initialize environment and client

As in previous lab, you will now set up the environment.

```
In [ ]: # =====
# Imports
# =====

# --- Third-party ---
from dotenv import load_dotenv
import aisuite as ai
import json

# --- Local / project ---
import utils
import display_functions
import email_tools

# =====
# Environment & Client
# =====
load_dotenv() # Load environment variables from .env
client = ai.Client() # Initialize AISuite client
```

3. Simulated email service

3.1 Components

This lab uses a **simulated email backend** to mimic real-world email interactions. Think of it as your personal sandbox email inbox: it comes preloaded with messages so you can practice without sending real emails.

Instead of building this backend from scratch, you'll be using tools that let you interact with it directly. Behind the scenes, it uses:

Layer	Purpose
FastAPI	Exposes REST endpoints
SQLite + SQLAlchemy	Stores and queries emails locally
Pydantic	Ensures inputs and outputs are valid
AI Suite tools	Bridge between the LLM and the service

3.2 Endpoints

The service provides several routes that simulate common email actions.

Later on, these will be wrapped as tools so the assistant can use them automatically:

- `POST /send` → send a new email
- `GET /emails` → list all emails
- `GET /emails/unread` → show only unread emails
- `GET /emails/{id}` → fetch a specific email by ID
- `GET /emails/search?q=...` → search emails by keyword
- `GET /emails/filter` → filter by recipient or date range
- `PATCH /emails/{id}/read` → mark an email as read
- `PATCH /emails/{id}/unread` → mark an email as unread
- `DELETE /emails/{id}` → delete an email by ID
- `GET /reset_database` → reset emails to initial state (for testing)

💡 **Key idea:** These endpoints act as the “inbox controls.”

In the next steps, you'll expose them as Python functions (tools) that the LLM can call—turning raw routes into agent actions.

3.3 Endpoint test helpers

Before giving control to the LLM, you'll first **test the backend yourself**. The `utils.test_*` functions are quick wrappers around the API endpoints. They let you try actions like **send, list, search, filter, mark, and delete** without writing raw HTTP requests.

Note: find the `utils.py` file from the top menu by navigating to `File > Open`

Each helper has a clear, self-explanatory name. In the next cell code, you can uncomment the one you want to run, check their output, and confirm that the email service behaves as expected.

For example, you can test:

- Send a test email
- Fetch email by ID
- List all messages
- Mark email as read or unread
- Delete email

This step is your **sanity check** before handing the controls over to the agent. It gives you confidence that the simulated email service is alive and behaving just like a real email service would.

👉 To test the endpoints, **uncomment** the lines you'd like to try out, run the cell, and check the results instantly.

```
In [ ]: # uncomment the line 'utils.test_*' you want to try
new_email_id = utils.test_send_email()
_ = utils.test_get_email(new_email_id['id'])
#_ = utils.test_list_emails()
#_ = utils.test_filter_emails(recipient="test@example.com")
#_ = utils.test_search_emails("lunch")
#_ = utils.test_unread_emails()
#_ = utils.test_mark_read(new_email_id['id'])
#_ = utils.test_mark_unread(new_email_id['id'])
#_ = utils.test_delete_email(new_email_id['id'])
#_ = utils.reset_database()
```

Output

POST /send: Status 200

```
{
  "id": 7,
  "sender": "you@mail.com",
  "recipient": "test@example.com",
  "subject": "Test Subject",
  "body": "This is a test email body.",
  "timestamp": "2026-03-28T07:25:41.400748",
  "read": false
}
```

GET /emails/7: Status 200

```
{
  "id": 7,
  "sender": "you@mail.com",
  "recipient": "test@example.com",
  "subject": "Test Subject",
  "body": "This is a test email body.",
  "timestamp": "2026-03-28T07:25:41.400748",
  "read": false
}
```

4. Tool layer for the email agent

4.1 Why tools?

Now that the endpoints are working, the next step is to expose them to the LLM as Python functions called **tools**. Each tool wraps a REST route, transforming raw API calls into actions the agent can perform —like list, read, search, send, delete, or toggle read.

Think of tools as the agent's **actuators**: you give a natural language instruction (“check unread emails from my boss and send a polite reply”), and the model chooses **which tools** to call and **in what order** to complete the task.

4.2 Design hints

- Keep tool **docstrings** short, imperative, and specific to the action.
- Return **consistent, compact JSON** so the model can chain results.
- Prefer **one clear responsibility per tool** (single route, single effect).

4.3 Available tools

Tool Function	Action
<code>list_all_emails()</code>	Fetch all emails, newest first
<code>list_unread_emails()</code>	Retrieve only unread emails
<code>search_emails(query)</code>	Search by keyword in subject, body, or sender
<code>filter_emails(...)</code>	Filter by recipient and/or date range
<code>get_email(email_id)</code>	Fetch a specific email by ID
<code>mark_email_as_read(id)</code>	Mark an email as read
<code>mark_email_as_unread(id)</code>	Mark an email as unread
<code>send_email(...)</code>	Send a new (simulated) email
<code>delete_email(id)</code>	Delete an email by ID
<code>search_unread_from_sender(addr)</code>	Return unread emails from a given sender (e.g., <code>boss@email.com</code>)

Note: find the `email_tools.py` file from the top menu by navigating to File > Open.

For example, testing `list_all_emails()` tool:

```
all_emails = email_tools.list_all_emails()
```

Now, let's try out a few tools that connect to the simulated endpoints to make sure everything is working.

👉 **Uncomment** the ones you'd like to run, execute the cell, and review the outputs.

```
# Test sending a new email and fetch it by ID
new_email = email_tools.send_email("test@example.com", "Lunch plans", "Shall we meet
at noon?")
content_ = email_tools.get_email(new_email['id'])

# Uncomment the ones you want to try:
#content_ = email_tools.list_all_emails()
#content_ = email_tools.list_unread_emails()
#content_ = email_tools.search_emails("lunch")
#content_ = email_tools.filter_emails(recipient="test@example.com")
#content_ = email_tools.mark_email_as_read(new_email['id'])
#content_ = email_tools.mark_email_as_unread(new_email['id'])
#content_ = email_tools.search_unread_from_sender("test@example.com")
#content_ = email_tools.delete_email(new_email['id'])

utils.print_html(content=json.dumps(content_, indent=2), title="Testing the
email_tools")
```

```
In [3]: # Test sending a new email and fetch it by ID
new_email = email_tools.send_email("test@example.com", "Lunch plan
content_ = email_tools.get_email(new_email['id'])

# Uncomment the ones you want to try:
#content_ = email_tools.list_all_emails()
#content_ = email_tools.list_unread_emails()
#content_ = email_tools.search_emails("lunch")
#content_ = email_tools.filter_emails(recipient="test@example.com")
#content_ = email_tools.mark_email_as_read(new_email['id'])
#content_ = email_tools.mark_email_as_unread(new_email['id'])
#content_ = email_tools.search_unread_from_sender("test@example.co
#content_ = email_tools.delete_email(new_email['id'])

utils.print_html(content=json.dumps(content_, indent=2), title="Te
```

Testing the email_tools

```
{
  "id": 8,
  "sender": "you@mail.com",
  "recipient": "test@example.com",
  "subject": "Lunch plans",
  "body": "Shall we meet at noon?",
  "timestamp": "2026-03-28T07:25:42.721608",
  "read": false
}
```

5. Preparing the agent prompt

Before assigning tasks to the email assistant agent, you'll create a small helper function called `build_prompt()`. This function wraps the natural language request in a system-style preamble so the LLM:

- Recognizes that it's acting as an **email assistant agent**
- Understands it has permission to use the available tools
- Executes actions directly, without asking for confirmation (no human-in-the-loop)

```
In [12]: def build_prompt(request_: str) -> str:
          return f"""
          - You are an AI assistant specialized in managing emails.
          - You can perform various actions such as listing, searching, filt
          - Use the provided tools to interact with the email system.
          - Never ask the user for confirmation before performing an action.
          - If needed, my email address is "you@email.com" so you can use it

          {request_.strip()}
          """
```

```
def build_prompt(request_: str) -> str:
    return f"""
    - You are an AI assistant specialized in managing emails.
```

- You can perform various actions such as listing, searching, filtering, and manipulating emails.
- Use the provided tools to interact with the email system.
- Never ask the user for confirmation before performing an action.
- If needed, my email address is "you@email.com" so you can use it to send emails or perform actions related to my account.

```
{request_.strip()}  
.....
```

```
example_prompt = build_prompt("Delete the Happy Hour email")  
utils.print_html(content=example_prompt, title="Example example_prompt")
```

Example example_prompt

- You are an AI assistant specialized in managing emails.
- You can perform various actions such as listing, searching, filtering, and manipulating emails.
- Use the provided tools to interact with the email system.
- Never ask the user for confirmation before performing an action.
- If needed, my email address is "you@email.com" so you can use it to send emails or perform actions related to my account.

Delete the Happy Hour email

5.3 Resetting the email service

Since you've been experimenting with the email service, let's reset it back to its initial state.

You can do this by calling the utility function that clears and refreshes the simulated email service:

```
In [6]: utils.reset_database()
```

```
{'message': 'Database reset and emails reloaded'}
```

6. LLM + Email tools

6.1 Scenario

So far, you've been working directly with the backend. Now it's time to let the **LLM take control** as your email assistant agent.

For example, you might ask it:

```
"Check for unread emails from boss@email.com , mark them as read, and send a polite follow-up."
```

6.2 What happen

1. The agent interprets your instruction.
2. It selects the right tools (`search_unread_from_sender` → `mark_email_as_read` → `send_email`).
3. It executes each action automatically, without asking for confirmation.

AlSuite handles schema exposure, argument binding, execution, and passing results between steps—so you can focus on **what** the agent achieves, not **how** to call the API.

6.3 Run it

Run the next cell and see how the agent orchestrates multiple tools to fulfill your request. Feel free to try your own requests with the email service as well.

What to look for:

- A clear **tool-call trace** showing which tools were used and the arguments passed.
- A concise **final message** summarizing the actions performed (e.g., "Found 1 unread email, marked as read, sent follow-up").

```
# Try your own requests
prompt_ = build_prompt("Check for unread emails from boss@email.com, mark them as read, and send a polite follow-up.")

response = client.chat.completions.create(
    model="openai:gpt-4.1", # LLM
    messages=[{"role": "user", "content": (
        prompt_
    )}],
```

```

tools=[ # list of tools that the LLM can access
    email_tools.search_unread_from_sender,
    email_tools.list_unread_emails,
    email_tools.search_emails,
    email_tools.get_email,
    email_tools.mark_email_as_read,
    email_tools.send_email
],
max_turns=5,
)

display_functions.pretty_print_chat_completion(response)

```

6.4. Missing tool: `delete_email`

What happens if the tool you need isn't available?

For example, try this request:

"Delete alice@work.com's email."

Since the `delete_email` tool is not registered, the LLM will still attempt to respond, but it won't be able to complete the action.

Key Insight

This highlights an important point: **the tools you make available directly determine what the agent can do.**

Available tools

Tool Function	Action
<code>list_all_emails()</code>	Fetch all emails, newest first
<code>list_unread_emails()</code>	Retrieve only unread emails
<code>search_emails(query)</code>	Search by keyword in subject, body, or sender
<code>filter_emails(...)</code>	Filter by recipient and/or date range
<code>get_email(email_id)</code>	Fetch a specific email by ID
<code>mark_email_as_read(id)</code>	Mark an email as read
<code>mark_email_as_unread(id)</code>	Mark an email as unread
<code>send_email(...)</code>	Send a new (mock) email
<code>delete_email(id)</code>	Delete an email by ID
<code>search_unread_from_sender(addr)</code>	Return unread emails from a given sender (e.g., <code>boss@email.com</code>)

```

# Try with a request that may call an unavailable tool
prompt_ = build_prompt("Delete alice@work.com email")

```

```

response = client.chat.completions.create(
    model="openai:o4-mini",
    messages=[{"role": "user", "content": (
        prompt_

```

```

    }},
    tools=[ # list of tools that the LLM can access
        email_tools.search_unread_from_sender,
        email_tools.list_unread_emails,
        email_tools.search_emails,
        email_tools.get_email,
        email_tools.mark_email_as_read,
        email_tools.send_email
    ],
    max_turns=5
)

display_functions.pretty_print_chat_completion(response)

```

 LLM Action: search_emails

```

{
  "query": "alice@work.com"
}

```

 Tool Response: search_emails

```

[
  {
    "id": 5,
    "sender": "alice@work.com",
    "recipient": "you@email.com",
    "subject": "Lunch?",
    "body": "Free for lunch today?",
    "timestamp": "2026-03-28T07:25:44.277140",
    "read": false
  }
]

```

 Final Assistant Message:

I found one email from alice@work.com (ID: 5, subject "Lunch?"). Unfortunately, the current system doesn't support deleting emails. Would you like me to mark it as read instead?

 Tool Sequence:

search_emails

6.4.1 Try again with `delete_email` enabled

What happens if the tool you need isn't available?

In the previous step of the lab, you gave the LLM access to the following list of tools:

```
tools=[
    email_tools.search_unread_from_sender,
    email_tools.list_unread_emails,
    email_tools.search_emails,
    email_tools.get_email,
    email_tools.mark_email_as_read,
    email_tools.send_email
]
```

And because of the tools available in that list, the agent couldn't delete emails because the `delete_email` tool wasn't available.

Now add `delete_email` to the tool list and re-run the cell. This time, the agent has everything it needs to finish the task.

Tip: Watch the sequence of calls — after finding the target message, the agent should select `delete_email` to complete the action.

```
prompt_ = build_prompt("Delete alice@work.com email")

response = client.chat.completions.create(
    model="openai:o4-mini",
    messages=[{"role": "user", "content": (
        prompt_
    )}],
    tools=[
        email_tools.search_unread_from_sender,
        email_tools.list_unread_emails,
        email_tools.search_emails,
        email_tools.get_email,
        email_tools.mark_email_as_read,
        email_tools.send_email,
        email_tools.delete_email
    ],
    max_turns=5
)

display_functions.pretty_print_chat_completion(response)
```

 LLM Action: search_emails

```
{
  "query": "alice@work.com"
}
```

 Tool Response: search_emails

```
[
  {
    "id": 5,
    "sender": "alice@work.com",
    "recipient": "you@email.com",
    "subject": "Lunch?",
    "body": "Free for lunch today?",
    "timestamp": "2026-03-28T07:25:44.277140",
    "read": false
  }
]
```

 LLM Action: delete_email

```
{
  "email_id": 5
}
```

 Tool Response: delete_email

```
{
  "message": "Email deleted"
}
```

 Final Assistant Message:

The email from alice@work.com has been deleted. Let me know if there's anything else you'd like to manage!

 Tool Sequence:

search_emails → delete_email